

Forward Propagation ও Backpropagation

ভূমিকা

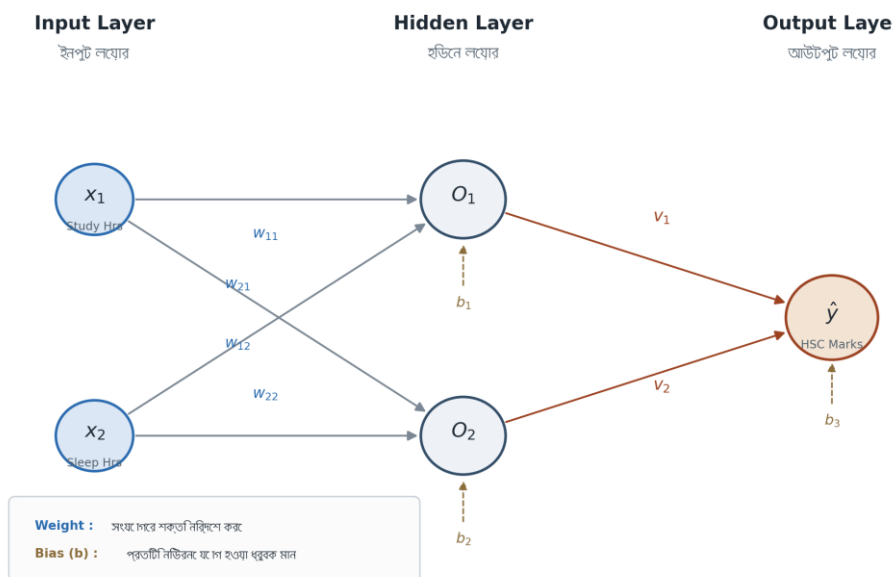
যেকোনো *Neural Network* মূলত দুটি ধাপে শেখে। প্রথমে **Forward Propagation**-এর মাধ্যমে ইনপুট থেকে একটি *Prediction* তৈরি হয়। এরপর **Backpropagation** ব্যবহার করে সেই *Prediction*-এর ভুল থেকে প্রতিটি *Parameter* (Weight ও Bias)-এর *Gradient* বের করা হয়। এই নোটে একটি ছোট *Regression* উদাহরণ ব্যবহার করে পুরো *Mathematical Process*টি ধাপে ধাপে হাতে-কলমে দেখানো হয়েছে, যাতে যেকোনো ছোট *Neural Network* নিজে হাতে হিসাব করে বোঝা যায়।

নেটওয়ার্ক আর্কিটেকচার ও উদাহরণ

আমরা একটি $2 \rightarrow 2 \rightarrow 1$ আর্কিটেকচারের *Neural Network* ব্যবহার করবো — অর্থাৎ 2টি *Input Feature*, 1টি *Hidden Layer* (2টি *Neuron* সহ), এবং 1টি *Output Neuron*। লক্ষ্য হলো Study Hours ও Sleep Hours থেকে একজন শিক্ষার্থীর HSC Marks পূর্বাভাস (Prediction) করা — এটি একটি সাধারণ *Regression* সমস্যা।

রাশি	অর্থ	মান
x_1	Study Hours (ইনপুট ১)	6
x_2	Sleep Hours (ইনপুট ২)	7
y	প্রকৃত HSC Marks (Actual)	80

Diagram ১: নেটওয়ার্ক আর্কিটেকচার



Weight ও Bias নোটেশন

সম্পূর্ণ নোট জুড়ে নিচের নোটেশন সামঞ্জস্যপূর্ণভাবে ব্যবহার করা হবে —

প্রতীক	সংযোগ / অবস্থান	প্রাথমিক মান
w_{11}	$x_1 \rightarrow O_1$	0.5
w_{21}	$x_2 \rightarrow O_1$	0.2
w_{12}	$x_1 \rightarrow O_2$	0.3

প্রতীক	সংযোগ / অবস্থান	প্রাথমিক মান
w_{22}	$x_2 \rightarrow O_2$	0.4
b_1	O_1 -এর Bias	0.1
b_2	O_2 -এর Bias	0.2
v_1	$O_1 \rightarrow \hat{y}$	10
v_2	$O_2 \rightarrow \hat{y}$	8
b_3	Output Neuron-এর Bias	1

এছাড়া *Learning Rate* হিসেবে $\eta = 0.001$ ব্যবহার করা হবে। এই হাতে-কলমে হিসাবের উদাহরণে ইচ্ছাকৃতভাবে সব **Weight শূন্য (zero) রাখা হয়নি** — কারণ সব Weight শূন্য হলে *Hidden Neuron* গুলো (O_1 ও O_2) একই মান পেত এবং তাদের Gradient-ও একই হতো। ফলে নেটওয়ার্ক আলাদা আলাদা *Feature* শিখতে পারতো না — একেই *Symmetry Problem* বলা হয়। তাই ভিন্ন ভিন্ন অ-শূন্য (non-zero) মান দিয়ে শুরু করা হয়েছে।

Forward Propagation

Forward Propagation মানে হলো — তথ্য *Input Layer* থেকে শুরু করে *Hidden Layer* হয়ে *Output Layer* পর্যন্ত এগিয়ে যাওয়া, যার ফলাফল হিসেবে একটি *Prediction* ($\hat{y}$) তৈরি হয়। প্রতিটি *Neuron*-এ দুটি কাজ ধারাবাহিকভাবে হয় — প্রথমে *Weighted Sum* হিসাব করা হয়, তারপর তার উপর একটি *Activation Function* প্রয়োগ করা হয়।

ধাপ ১ — Hidden Neuron O_1 গণনা

প্রথমে O_1 -এর *Weighted Sum* (z_1) বের করতে হবে —

$$z_1 = x_1 w_{11} + x_2 w_{21} + b_1$$

মান বসিয়ে —

$$z_1 = (6)(0.5) + (7)(0.2) + 0.1$$

$$z_1 = 3 + 1.4 + 0.1 = \mathbf{4.5}$$

এখন এর উপর **ReLU** (Rectified Linear Unit) *Activation Function* প্রয়োগ করা হয়। ReLU-এর সংজ্ঞা হলো —

$$\text{ReLU}(z) = \max(0, z)$$

অর্থাৎ মান ধনাত্মক (positive) হলে তা অপরিবর্তিত থাকে, ঋণাত্মক (negative) হলে তা শূন্য হয়ে যায়। যেহেতু $z_1 = 4.5$ ধনাত্মক, তাই —

$$O_1 = \text{ReLU}(4.5) = \mathbf{4.5}$$

ধাপ ২ — Hidden Neuron O_2 গণনা

একইভাবে দ্বিতীয় Hidden Neuron-এর জন্য —

$$z_2 = x_1 w_{12} + x_2 w_{22} + b_2$$

$$z_2 = (6)(0.3) + (7)(0.4) + 0.2 = 1.8 + 2.8 + 0.2 = \mathbf{4.8}$$

$$O_2 = \text{ReLU}(4.8) = \mathbf{4.8}$$

ধাপ ৩ — চূড়ান্ত Output গণনা

যেহেতু এটি একটি *Regression* সমস্যা, তাই *Output Layer*-এ কোনো *Activation* ব্যবহার না করে সরাসরি (Linear) মান বের করা হয় —

$$\hat{y} = O_1 v_1 + O_2 v_2 + b_3$$

$$\hat{y} = (4.5)(10) + (4.8)(8) + 1 = 45 + 38.4 + 1$$

$$\hat{y} = 84.4$$

অর্থাৎ, **প্রকৃত মান (Actual) = 80** এবং **পূর্বাভাস (Prediction) = 84.4**। যেহেতু দুটি মান সমান নয়, তাই মডেলের মধ্যে একটি *Error* রয়ে গেছে — এই ভুলের পরিমাণ কতটা, তা মাপার জন্য পরের ধাপে *Loss Function* ব্যবহার করা হবে।

Loss গণনা

Prediction কতটা ভুল হয়েছে তা পরিমাপ করতে *Loss Function* ব্যবহার করা হয়। এই *Regression* সমস্যার জন্য **Mean Squared Error**-এর একটি সরল রূপ ব্যবহার করা হচ্ছে —

$$L = \frac{1}{2}(\hat{y} - y)^2$$

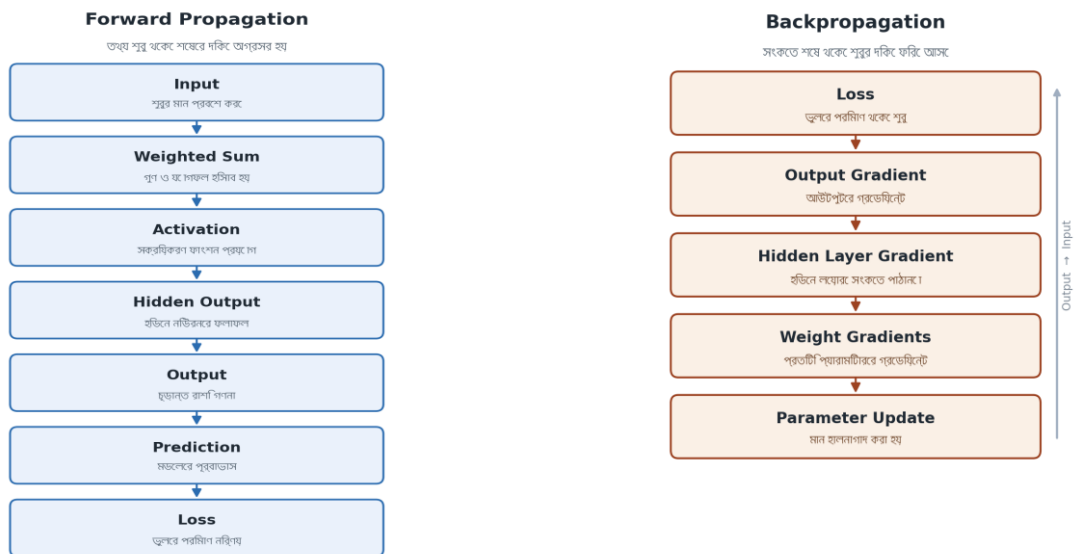
এখানে $\frac{1}{2}$ ব্যবহার করার কারণ হলো — যখন আমরা এই *Loss*-কে পরে *Derivative* করবো, তখন বর্গ (square) থেকে আসা 2 এই $\frac{1}{2}$ -এর সাথে কাটাকাটি হয়ে গিয়ে হিসাব সহজ হয়ে যায়। মান বসিয়ে —

$$L = \frac{1}{2}(84.4 - 80)^2$$

$$L = \frac{1}{2}(4.4)^2 = 9.68$$

Forward Propagation থেকে Backpropagation-এ যাত্রা

Forward Propagation আমাদের বলে দেয় মডেল কী *Predict* করলো। *Loss* বলে দেয় সেই Prediction কতটা ভুল ছিল। আর **Backpropagation** বলে দেয় — সেই ভুলের জন্য প্রতিটি *Parameter* (Weight/Bias) ঠিক কতটা দায়ী।



Backpropagation

Backpropagation হলো *Loss* থেকে পিছনের দিকে গিয়ে প্রতিটি *Weight* ও *Bias*-এর *Gradient* বের করার *Process*। এখানে ভুল বোঝার একটি সাধারণ জায়গা পরিষ্কার করা দরকার — **Backpropagation** মানে তথ্য (data)

পিছনের দিকে সরানো নয়; বরং এটি *Computational Graph*-এর মধ্য দিয়ে **Error Signal** পিছনের দিকে প্রচার করে প্রতিটি Parameter-এর Gradient হিসাব করার পদ্ধতি।

এই পুরো হিসাবের ভিত্তি হলো **Chain Rule**।

Chain Rule — ধারণা

ধরা যাক চারটি রাশি এভাবে একে অপরকে প্রভাবিত করে —

$$A \rightarrow B \rightarrow C \rightarrow D$$

যদি A প্রভাবিত করে B -কে, B প্রভাবিত করে C -কে, এবং C প্রভাবিত করে D -কে, তাহলে A আসলে D -এর উপর কতটা প্রভাব ফেলে তা বের করতে পথ ধরে প্রতিটি প্রভাব একসাথে গুণ করতে হয়। *Neural Network*-এ একইভাবে —

$$w_{11} \rightarrow O_1 \rightarrow \hat{y} \rightarrow L$$

তাই —

$$\partial L / \partial w_{11} = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial O_1 \times \partial O_1 / \partial w_{11}$$

এখানে প্রতিটি অংশের অর্থ —

$\partial L / \partial \hat{y}$	Loss-এর মান Output ($\hat{y}$) বদলালে কতটা বদলায়
$\partial \hat{y} / \partial O_1$	Output ($\hat{y}$)-এর মান O_1 বদলালে কতটা বদলায়
$\partial O_1 / \partial w_{11}$	O_1 -এর মান w_{11} বদলালে কতটা বদলায়

Backpropagation-এর গাণিতিক হিসাব

ধাপ ১ — $\partial L / \partial \hat{y}$ নির্ণয়

আমরা জানি —

$$L = 1/2(\hat{y} - y)^2$$

এটিকে $\hat{y}$ -এর সাপেক্ষে Derivative করলে (Power Rule অনুযায়ী, বর্গের সূচক 2 নেমে এসে $1/2$ -কে কেটে দেয়) —

$$\partial L / \partial \hat{y} = \hat{y} - y$$

$$\partial L / \partial \hat{y} = 84.4 - 80 = 4.4$$

এই 4.4 সংখ্যাটির অর্থ হলো — Prediction ($\hat{y}$) সামান্য বাড়ালে Loss কতটা বাড়বে তার হার। এই মানই পুরো Backpropagation-এর সূচনা বিন্দু কারণ এখান থেকেই Error Signal নেটওয়ার্কের বাকি অংশে ছড়িয়ে পড়বে।

ধাপ ২ — Output Layer-এর Gradient (v_1, v_2, b_3)

v_1 -এর জন্য Chain Rule প্রয়োগ করে —

$$\partial L / \partial v_1 = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial v_1$$

যেহেতু $\hat{y} = O_1 v_1 + O_2 v_2 + b_3$, তাই $\hat{y}$ -কে v_1 -এর সাপেক্ষে Derivative করলে —

$$\partial \hat{y} / \partial v_1 = O_1$$

অতএব —

$$\partial L / \partial v_1 = 4.4 \times 4.5 = \mathbf{19.8}$$

একইভাবে v_2 -এর জন্য —

$$\partial L / \partial v_2 = 4.4 \times 4.8 = \mathbf{21.12}$$

এবং b_3 -এর জন্য —

$$\partial L / \partial b_3 = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial b_3$$

যেহেতু $\hat{y}$ -এর সূত্রে b_3 -এর সহগ (coefficient) হলো 1, তাই $\partial \hat{y} / \partial b_3 = 1$ — অর্থাৎ যেকোনো Bias-এর Local Derivative সবসময় 1 হয়, কারণ Bias সরাসরি যোগ (add) হয়, কোনো কিছু দিয়ে গুণ হয় না।

$$\partial L / \partial b_3 = 4.4 \times 1 = \mathbf{4.4}$$

ধাপ ৩ — Hidden Layer-এ Error Signal প্রবাহ

এখন Error Signal Output থেকে Hidden Layer-এর দিকে (O_1 ও O_2 -এর দিকে) ফিরে যাবে। O_1 -এর জন্য —

$$\partial L / \partial O_1 = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial O_1$$

যেহেতু $\partial \hat{y} / \partial O_1 = v_1$ (কারণ $\hat{y}$ -এর সূত্রে O_1 -এর সহগ হলো v_1), তাই —

$$\partial L / \partial O_1 = 4.4 \times 10 = \mathbf{44}$$

একইভাবে O_2 -এর জন্য —

$$\partial L / \partial O_2 = 4.4 \times 8 = \mathbf{35.2}$$

ধাপ ৪ — ReLU-এর Derivative

O_1 ও O_2 থেকে সরাসরি w -এ যাওয়ার আগে ReLU Activation-এর ভেতর দিয়ে যেতে হয়, তাই ReLU-এর Derivative জানা দরকার। ReLU-এর সংজ্ঞা ছিল —

$$\text{ReLU}(z) = \max(0, z)$$

এর Derivative হলো —

- যখন $z > 0$ — তখন $\text{ReLU}'(z) = \mathbf{1}$ (কারণ ঢাল/slope সরলরেখার মতো 1)
- যখন $z < 0$ — তখন $\text{ReLU}'(z) = \mathbf{0}$ (কারণ ঐ অংশে মান সবসময় শূন্য, তাই কোনো পরিবর্তন নেই)

এই Activation Derivative-টি Chain Rule-এর একটি জরুরি অংশ — এটি ছাড়া Error Signal Weighted Sum (z) পর্যন্ত পৌঁছাতে পারতো না। যেহেতু আমাদের $z_1 = 4.5 > 0$ এবং $z_2 = 4.8 > 0$, তাই দুটোরই Derivative **1**।

$$\partial L / \partial z_1 = \partial L / \partial O_1 \times \text{ReLU}'(z_1) = 44 \times 1 = \mathbf{44}$$

$$\partial L / \partial z_2 = \partial L / \partial O_2 \times \text{ReLU}'(z_2) = 35.2 \times 1 = \mathbf{35.2}$$

ধাপ ৫ — Input → Hidden Weight-গুলোর Gradient

এখন সম্পূর্ণ Chain দেখা যাক w_{11} -এর জন্য —

$$w_{11} \rightarrow z_1 \rightarrow O_1 \rightarrow \hat{y} \rightarrow L$$

$$\partial L / \partial w_{11} = \partial L / \partial z_1 \times \partial z_1 / \partial w_{11}$$

যেহেতু $z_1 = x_1 w_{11} + x_2 w_{21} + b_1$, তাই w_{11} -এর সাপেক্ষে z_1 -এর Derivative হলো শুধু তার সহগ x_1 —

$$\partial z_1 / \partial w_{11} = x_1$$

$$\partial L / \partial w_{11} = 44 \times 6 = \mathbf{264}$$

অর্থাৎ এই *Training Example*-এর ক্ষেত্রে w_{11} -এর Gradient হলো **264**। এই Gradient ব্যবহার করে পরবর্তী ধাপে Optimizer w_{11} Update করবে।

একইভাবে বাকি তিনটি Weight-এর জন্য —

$$\partial L / \partial w_{21} = \partial L / \partial z_1 \times x_2 = 44 \times 7 = \mathbf{308}$$

w_{12} ও w_{22} -এর জন্য প্রথমে z_2 -এর Chain ব্যবহার হবে —

$$\partial L / \partial w_{12} = \partial L / \partial z_2 \times x_1 = 35.2 \times 6 = \mathbf{211.2}$$

$$\partial L / \partial w_{22} = \partial L / \partial z_2 \times x_2 = 35.2 \times 7 = \mathbf{246.4}$$

ধাপ ৬ — Bias-গুলোর Gradient

যেহেতু $z = (\text{weighted sum}) + b$, তাই যেকোনো Bias-এর সাপেক্ষে z -এর Derivative সবসময় **1** —

$$\partial z / \partial b = 1$$

তাই Bias-এর Gradient সরাসরি সেই নিউরনের Upstream Gradient-এর সমান হয়ে যায় —

$$\partial L / \partial b_1 = \partial L / \partial z_1 \times 1 = \mathbf{44}$$

$$\partial L / \partial b_2 = \partial L / \partial z_2 \times 1 = \mathbf{35.2}$$

সম্পূর্ণ Gradient Table

Parameter	Gradient ($\partial L / \partial \text{param}$)
w_{11}	264
w_{21}	308
w_{12}	211.2
w_{22}	246.4
b_1	44
b_2	35.2
v_1	19.8
v_2	21.12
b_3	4.4

Backpropagation শুধু এই **Gradient-গুলো** হিসাব করে — এটি নিজে থেকে কোনো Parameter **Update** করে না।

Parameter Update — Gradient Descent

Backpropagation-এর হিসাব করা Gradient ব্যবহার করে প্রকৃত Update করে **Gradient Descent** নামের Optimizer। নিয়মটি হলো —

$$w_{new} = w_{old} - \eta \times \frac{\partial L}{\partial w}$$

এবং একইভাবে Bias-এর জন্য —

$$b_{new} = b_{old} - \eta \times \frac{\partial L}{\partial b}$$

এখানে $\eta = 0.001$ — অর্থাৎ প্রতিবার খুবই ছোট পদক্ষেপে (small step) Parameter আপডেট হয়, যাতে Loss ধীরে ধীরে কমতে থাকে। কয়েকটি উদাহরণ দেখা যাক —

$$w_{11new} = 0.5 - (0.001 \times 264) = 0.5 - 0.264 = \mathbf{0.236}$$

$$w_{21new} = 0.2 - (0.001 \times 308) = 0.2 - 0.308 = \mathbf{-0.108}$$

$$b_{1new} = 0.1 - (0.001 \times 44) = 0.1 - 0.044 = \mathbf{0.056}$$

লক্ষ্য করার বিষয় — w_{21} এখন ঋণাত্মক (negative) হয়ে গেছে। এটি স্বাভাবিক; Training চলতে থাকলে Weight ধনাত্মক বা ঋণাত্মক যেকোনো দিকে যেতে পারে, যেভাবে Loss কমানো সম্ভব হয়।

Update-এর পর সম্পূর্ণ Parameter Table

Parameter	পুরনো মান	নতুন মান
w_{11}	0.5	0.236
w_{21}	0.2	-0.108
w_{12}	0.3	0.0888
w_{22}	0.4	0.1536
b_1	0.1	0.056
b_2	0.2	0.1648
v_1	10	9.9802
v_2	8	7.97888
b_3	1	0.9956

গুরুত্বপূর্ণ ধারণাগত পার্থক্য

Forward Propagation

প্রশ্ন: মডেল কী Predict করলো?

Input → Hidden → Output

Loss

প্রশ্ন: Prediction কতটা ভুল হলো?

$L(y, \hat{y})$

Backpropagation

প্রশ্ন: প্রতিটি Parameter ভুলের জন্য কতটা দায়ী?

$\frac{\partial L}{\partial w}$

Optimizer (Gradient Descent)

প্রশ্ন: Parameter-গুলো কীভাবে বদলানো উচিত?

$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$

সাধারণ কিছু ভুল ধারণা পরিষ্কার করা

১. Input Node কোনো গাণিতিক Neuron নয়

Input Layer মূলত শুধু Input Feature-গুলো (x_1, x_2) ধারণ করে — এখানে কোনো Weighted Sum বা Activation হিসাব হয় না।

২. একটি Input একাধিক Neuron-এর সাথে যুক্ত থাকতে পারে

যেমন — $x_1 \rightarrow O_1$ এবং একই সাথে $x_1 \rightarrow O_2$ । একটি Input একাধিক পথে নেটওয়ার্কে প্রভাব ফেলতে পারে।

৩. Backpropagation মানে ডেটা পিছনে সরানো নয়

এর অর্থ হলো Computational Graph-এর মধ্য দিয়ে Error Signal পিছনের দিকে প্রচার করে প্রতিটি Parameter-এর Gradient হিসাব করা — ডেটা পয়েন্ট (x_1, x_2) নিজে কোথাও সরে না।

৪. Backpropagation ও Gradient Descent এক জিনিস নয়

Backpropagation Gradient হিসাব করে।	Gradient Descent Parameter আপডেট করে।
---	---

সংক্ষিপ্ত পুনরালোচনা (Final Recap)

Input → Weighted Sum → Activation → Hidden Output → Prediction → Loss
Loss → Gradient → Chain Rule → Backpropagation → Parameter Gradients → Update

Neural Network প্রথমে *Forward Propagation*-এর মাধ্যমে *Prediction* তৈরি করে। এরপর *Loss Function* Prediction-এর ভুল পরিমাপ করে। **Backpropagation** *Chain Rule* ব্যবহার করে Loss থেকে পিছনের দিকে গিয়ে প্রতিটি Weight ও Bias-এর *Gradient* বের করে। এরপর *Optimizer* সেই Gradient ব্যবহার করে Parameters Update করে। এই *Process* বারবার চলার মাধ্যমে Network ধীরে ধীরে ভালো *Prediction* করতে শেখে।